

Theory of Computation BCSE0404X

Regular Expression and Tokenization

B. Tech
(Computer Science & Engineering)
4th Semester

Vivek Kumar Sharma
Assistant Professor
Computer Science and
Engineering



Faculty Introduction

Vivek Kumar Sharma is an Assistant Professor at Computer Science and Engineering Department, Noida Institute of Engineering and Technology, Greater Noida. He has done M. Tech. in 2013 and B.Tech in 2010. He is currently pursuing Ph.D. in Image Security.

Evaluation Scheme

**NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY, GREATER NOIDA, GAUTAM BUDDH NAGAR
(AN AUTONOMOUS INSTITUTE)**

**Bachelor of Technology
Computer Science & Engineering
Evaluation Scheme
SEMESTER-IV**

Sl. No.	Subject Codes	Subject	Types of Subjects	Periods			Evaluation Schemes				End Semester		Total	Credit
				L	T	P	CT	TA	TOTAL	PS	TE	PE		
1	BASCC0401	Employability Skill Development - II	Mandatory	2	0	0	60	40	100				100	2
2	BCSE0402	Database Management Systems	Mandatory	3	0	0	30	20	50		100		150	3
3	BCSE0401	Data Structures and Algorithms-II	Mandatory	3	0	0	30	20	50		100		150	3
4	BCSE0404X	Theory of Computation	Mandatory	3	0	0	30	20	50		100		150	3
5		Department Elective - I	Departmental Elective	3	0	0	30	20	50		100		150	3
6	BAS0403N	Statistics and Probability	Mandatory	3	1	0	30	20	50		100		150	4
7	BCSE0452Z	Database Management Systems Lab	Mandatory	0	0	4				50		50	100	2
8	BCSE0451	Data Structures and Algorithms-II Lab	Mandatory	0	0	2				25		25	50	1
9	BCSE0455	Web Technologies	Mandatory	0	0	6				50		100	150	3
10	BCSE0459	Mini Project	Mandatory	0	0	2				50			50	1
11	BCSCC0452	Problem Solving Approaches	Mandatory	0	0	2				50			50	1
12	BNC0402Y/ BNC0401Y	Environmental Science / Artificial Intelligence and Cyber Ethics	Compulsory Audit	2	0	0	30	20	50				50	NA
		*Massive Open Online Courses (For B.Tech. Hons. Degree)	*MOOCs											
		TOTAL		20	1	16			350	225	500	175	1250	26

Syllabus

Course Contents / Syllabus		
Module 1	Introduction to Finite Automata and Compiler	10 hours
Introduction: Role of Automata and Formal languages, Alphabet, String, Grammar, Language, Chomsky Hierarchy of languages. Introduction to Finite State Machine: Deterministic Finite Automaton (DFA) and Non-Deterministic Finite Automaton (NFA), NFA with ϵ -Transition, Equivalence of NFA and DFA, Introduction to Compiler: Translators, Language Processing System, Phases and passes of compilation Use Case 1 : Role of Finite Automata in NLP and Speech Recognition.		
Module 2	Regular Expression and Tokenization	9 hours
Regular Expression: Regular Expression, Regular Sets, Properties of Regular Expression, Finite Automata and Regular Expression, Arden's Theorem, Regular Grammars-Right Linear and Left Linear grammars. Lexical Analyzer: Role of lexical Analyzer, Specifications and Recognition of tokens, Lex,		
Module 3	Context free grammar and Push Down Automata	09 hours
Context Free Grammar (CFG): Definition and Language, Derivations, Parse Trees and Ambiguity, Simplification of CFG, Push Down Automata: Definition of the Pushdown Automata, Languages of PDA		
Module 4	Parser and Intermediate Representation	10 hours
Parser: Role of parser, Top down Parsing-LL (1) parser, Bottom up parsing- shift reduce parser and LR (0), SLR parser, Introduction to Syntax directed Translation, Intermediate-Code Generation: Three-Address Code- Quadruples, Triples, Indirect triples Use Case-2: Role of CFG and parsing in Voice Assistant		
Module 5	Turing machine and Optimization	10 hours
Turing Machine: Basic Concept of Turing Machine, Variants of Turing Machine, Universal Turing Machine Code optimization and generation: Basic Block, Flow graph, DAG, Optimization Techniques		
Total Lecture Hours		48 hours

Computer Science

- Automaton is nothing but a machine which accepts the strings of a language L over an input alphabet Σ . There are four different types of Automata that are mostly used in various areas
- Lexical analysis and regular expression matching in compilers and search engines.
- Syntax parsing in programming language compilers and NLP.
- Finite State Machines in hardware design, network protocols, and robotics.
- Speech recognition using Hidden Markov Models (HMMs).
- Text processing in editors and search tools.
- Bioinformatics and genetic sequence analysis.

Course Objectives

The course objective of this course:

The primary objective of this course is to provide a foundational understanding of Automata Theory and its role in the language processing systems, also explores their application in fields like Natural Language Processing (NLP), speech recognition

Course Outcomes

CO	At the end of course , the student will be able to	Bloom's (KL)
CO1	Identify the fundamental concepts of automata theory, formal languages and compiler components	K ₂
CO2	Understand the role of finite automata, regular expressions, and grammar rules in language processing.	K ₂
CO3	Demonstrate context-free grammars, pushdown automata, and syntax-directed translation to construct intermediate code for language processors	K ₃
CO4	Analyze various parsing strategies, code translation methods, and intermediate representations in compiler phases.	K ₄
CO5	Analyze the functioning of Turing machine models and optimization techniques in code generation for performance improvement	K ₄

Program Outcomes (POs)

Engineering Graduates will be able to:

- 1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- 2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- 3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- 4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

Program Outcomes (POs)

Contd..

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

Program Outcomes (POs)

Contd..

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CO-PO correlation matrix

CO-PO Mapping	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
C01	3	2	2	2	-	-	-	-	-	-	2
C02	3	3	3	2	1	-	-	-	-	-	2
C03	3	3	3	3	1	-	-	-	-	-	2
C04	3	3	3	3	1	-	-	-	-	-	2
C05	3	3	3	3	1	-	-	-	-	-	2

Program Specific Outcomes

On successful completion of graduation degree the Computer Science & Engineering graduates will be able to:

PSO1:	identify, analyze real world problems and design their ethical solutions using artificial intelligence, robotics, virtual/augmented reality, data analytics, block chain technology, and cloud computing.
PSO2:	Design and develop the hardware sensor devices and related interfacing software systems for solving complex engineering problems.
PSO3:	understand inter-disciplinary computing techniques and to apply them in the design of advanced computing.
PSO4:	Conduct investigation of complex problem with the help of technical, managerial, leadership qualities, and modern engineering tools provided by industry sponsored laboratories.

CO-PSO correlation matrix

CO	PSO			
	PSO1	PSO2	PSO3	PSO4
CO1	3	2	3	2
CO2	3	2	3	2
CO3	3	2	3	2
CO4	3	2	3	2
CO5	3	2	3	2
Average	3	2	3	2

Program Educational Objectives

PEO'S of the B. Tech. in Computer Science and Engineering are:

- The graduates will be able:

PEO 1: To have an excellent scientific and engineering breadth so as to comprehend, analyze, design and provide sustainable solutions for real-life problems using state-of-the-art technologies.

PEO 2: To have a successful career in industries, to pursue higher studies or to support entrepreneurial endeavors and to face global challenges.

PEO 3: To have an effective communication skill, professional attitude, ethical values and a desire to learn specific knowledge in emerging trends, technologies for research, innovation and product development and contribution to society.

PEO 4: To have life-long learning for up-skilling and re-skilling for a successful professional career as an engineer, scientist, entrepreneur, or bureaucrat for the betterment of the society.

CO-PEO correlation matrix

CO	PEO			
	PEO1	PEO2	PEO3	PEO4
CO1	2	2	1	2
CO2	2	2	1	1
CO3	2	2	2	2
CO4	2	2	1	1
CO5	2	2	1	2
Average	2	2	1.2	2

Subject Result:

Section A: 95.65%

Section B: 97.14%

Section C: 84.51%

Section D: 94.29%

End Semester Question Paper Template

Total pages:

Subject Code:

NOIDA INSTITUTE OF ENGINEERING AND TECHNOLOGY, GREATER NOIDA, G.B NAGAR
(An Autonomous Institute Affiliated to AKTU, Lucknow) / (MCA Institute)

Course:

SEM: - THEORY EXAMINATION (20..... - 20.....)

SOLUTION

Subject Name

Total Marks:.....

General Instructions for Solution & Assessment of Answer sheets.

1. Assessment of answer books of semester end examination shall be done by appointed evaluators only.
2. Award step-wise marks for each solved question.
3. Marks to be awarded in the column provided.

+	SECTION – A		Awarded Marks
	1. Attempt all parts:- (SOLUTION)		10
	1-a.		
	1-b.		
	1-c.		
	1-d.		
	1-e.		
	1-f.		

End Semester Question Paper Template

2. Attempt all parts:- (SOLUTION)		10
2.a.	Your Subjective Question (CO1)	
2.b.	Your Subjective Question (CO2)	
2.c.	Your Subjective Question (CO3)	
2.d.	Your Subjective Question (CO4)	
2.e.	Your Subjective Question (CO5)	
SECTION – B		
3. (SOLUTION) of all questions		30
3-a.	Your Subjective Question (CO1)	
3-b.	Your Subjective Question (CO1)	
3-c.	Your Subjective Question (CO2)	

End Semester Question Paper Template

3-d.	Your Subjective Question (CO2)	
3-e.	Your Subjective Question (CO3)	
3-f.	Your Subjective Question (CO4)	
3-g.	Your Subjective Question (CO5)	
SECTION – C		
4.	(SOLUTION) of all questions	70
4-a.	Your Subjective Question (CO1)	
4-b.	Your Subjective Question (CO1)	
5.	(SOLUTION) of all questions	
5-a.	Your Subjective Question (CO2)	
5-b.	Your Subjective Question (CO2)	
6.	(SOLUTION) of all questions	
6-a.	Your Subjective Question (CO3)	
6-b.	Your Subjective Question (CO3)	
7.	(SOLUTION) of all questions	
7-a.	Your Subjective Question (CO4)	
7-b.	Your Subjective Question (CO4)	
8.	(SOLUTION) of all questions	
8-a.	Your Subjective Question (CO5)	
8-b.	Your Subjective Question (CO5)	

Prerequisite for the Course

- Basics operations of mathematics.
- Discrete mathematics.
- Predicate logic.

Recap

- Finite automata is an abstract computing device. It is a mathematical model of a system with discrete inputs, outputs, states and a set of transitions from state to state that occurs on input symbols from the alphabet Σ .

Brief subject introduction with Video Lecture

- [\(73\) TAFL1:Theory of Automata and Formal Language | Course Overview of automata, TOC Lectures in Hindi - YouTube](#)
- [\(73\) TAFL2:Theory of Automata and Formal Language | Theory of Computation Tutorial Syllabus - YouTube](#)

Unit Contents

Topics	Duration (in Hours)
Introduction	3
Introduction to Finite State Machine	4
Introduction to Compiler	3

Topics	Duration (in Hours)
Assignment, Tutorials and Quiz	1

Regular Expression(RE)

- We have already discussed that languages accepted by Finite Automata are called Regular Language.
- *Regular expressions* are an algebraic way to describe languages.
- They describe exactly the regular languages.
- If E is a regular expression, then $L(E)$ is the language it defines.
- We'll describe RE's and their languages recursively.
- An expression that express the regular language is called the regular Expression
- Any set that represents the value of the Regular Expression is called a **Regular Set**
- RE is written for the Regular Language using +, ., * is known as Regular Expression.

Examples

- $(a+b)$
- $(0+10^*)$
- $(a+\epsilon)b^*a$
- $(ab+ba)^*$

Regular Expression(RE)

Formally a regular expression is defined recursively as

1. Any terminal symbol (i.e. an element of Σ), ϵ and ϕ are regular expressions. When we view a in Σ as a regular expression, we **denote it by a** .
2. The union of two regular expressions R_1 and R_2 written as **$R_1 + R_2$, is also a regular expression.**
3. The concatenation of two regular expressions R_1 and R_2 , written as **$R_1 \cdot R_2$, is also a regular expression.**
4. The iteration (or closure) of a regular expression R written as **R^* , is also a regular expression.**
5. *If R is a regular expression, then **(R) is also a regular expression.***
6. The regular expressions over Σ are precisely those obtained recursively by the application of the rules 1-5 *once or several times.*

Regular Expression(RE)

Basis 1: If a is any symbol, then a is a RE, and $L(a) = \{a\}$.

Note: $\{a\}$ is the language containing one string, and that string is of length 1.

Basis 2: ϵ is a RE, and $L(\epsilon) = \{\epsilon\}$.

Basis 3: $\emptyset$ is a RE, and $L(\emptyset) = \emptyset$.

Induction 1: If E_1 and E_2 are regular expressions, then $E_1 + E_2$ is a regular expression, and $L(E_1 + E_2) = L(E_1) \cup L(E_2)$.

Induction 2: If E_1 and E_2 are regular expressions, then $E_1 E_2$ is a regular expression, and $L(E_1 E_2) = L(E_1) L(E_2)$.

Concatenation : the set of strings wx such that w is in $L(E_1)$ and x is in $L(E_2)$.

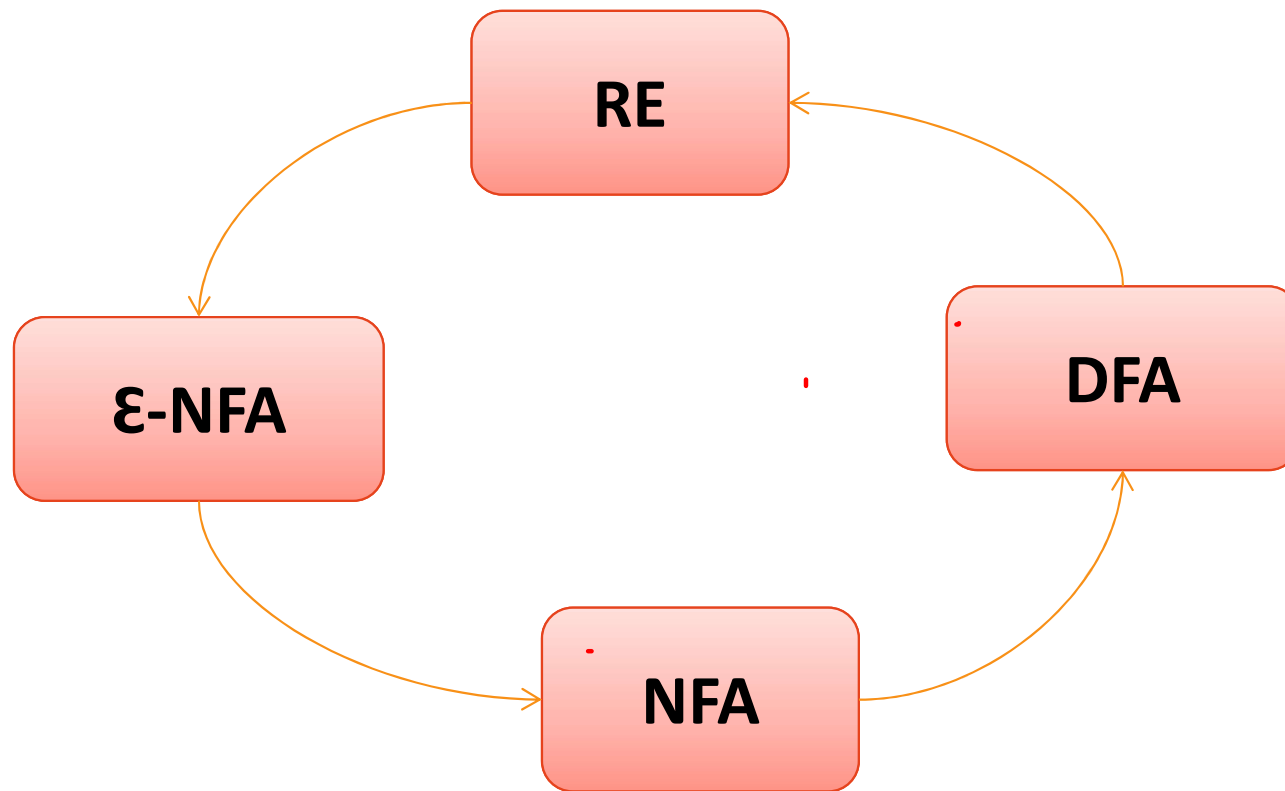
Regular Expression(RE)

Induction 3: If E is a RE, then E^* is a RE, and $L(E^*) = (L(E))^*$.

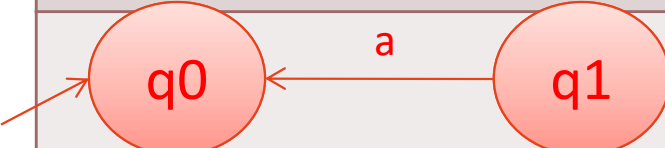
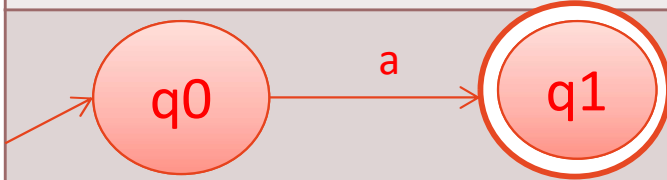
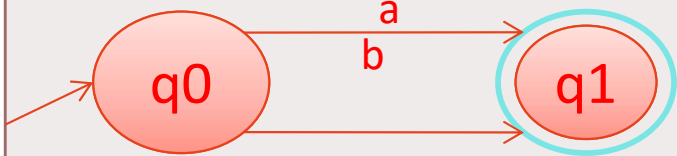
Closure, or “Kleene closure” = set of strings $w_1w_2...w_n$, for some $n \geq 0$, where each w_i is in $L(E)$.

Note: when $n=0$, the string is ϵ .

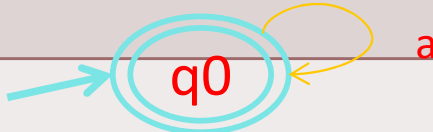
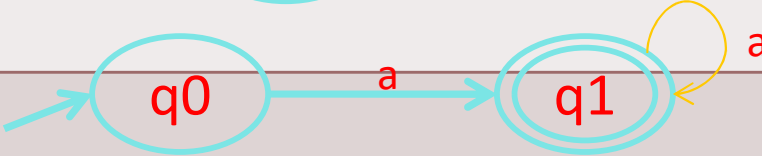
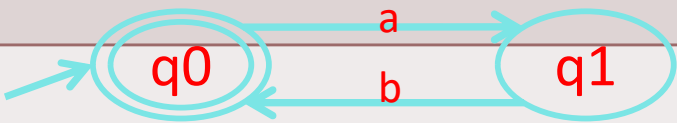
DFA, NFA, ϵ -NFA and Regular Expression(RE)



DFA, Regular Sets, Regular Expressions

Finite Automata	Regular Set	Regular Expression
	$\{\epsilon\}$	$RE = \epsilon$
	$\{\phi\}$	$RE = \phi$
	$\{a\}$	$RE = a$
	$\{a, b\}$	$RE = a+b$

DFA, Regular Sets, Regular Expressions

Finite Automata	Regular Set	Regular Expression
	{ab}	RE = a.b
	{ ϵ , a, aa, aaa,}	RE = a^*
	{a, aa, aaa,}	RE = $a.a^* = a^+$
	{ ϵ , ab, abab,}	RE = $(ab)^*$

Regular Expressions, Regular Sets

Regular Expressions	Regular Set
$(0 + 10^*)$	$L = \{ 0, 1, 10, 100, 1000, 10000, \dots \}$
(0^*10^*)	$L = \{1, 01, 10, 010, 0010, \dots\}$
$(0 + \epsilon)(1 + \epsilon)$	$L = \{\epsilon, 0, 1, 01\}$
$(a+b)^*$	$L = \{\epsilon, a, b, aa, ab, bb, ba, aaa, \dots\}$
$(a+b)^*abb$	$L = \{abb, aabb, babb, aaabb, ababb, \dots\}$ ending with abb
$(11)^*$	$L = \{\epsilon, 11, 1111, 111111, \dots\}$
$(aa)^*(bb)^*b$	Set of strings consisting of even number of a's followed by odd number of b's , so $L = \{b, aab, aabbb, aabbbbb, aaaab, aaaabbb, \dots\}$
$(aa + ab + ba + bb)^*$	String of a's and b's of even length can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null, so $L = \{\epsilon, aa, ab, ba, bb, aaab, aaba, \dots\}$

Examples of Regular Expression

Q1: Write the regular expression for the language accepting all combinations of a's, over the set $\Sigma = \{a\}$

Solution:

All combinations of a's means a may be zero, single, double and so on. If a is appearing zero times, that means a null string. That is we expect the set of $\{\epsilon, a, aa, aaa, \dots\}$. So we give a regular expression for this as:

$$R = a^*$$

That is Kleen closure of a.

Examples of Regular Expression

Q2: Write the regular expression for the language accepting all combinations of a's except the null string, over the set $\Sigma = \{a\}$

Solution:

The regular expression has to be built for the language

$$L = \{a, aa, aaa, \dots\}$$

This set indicates that there is no null string. So we can denote regular expression as: $R = a^+$

Q3: Write the regular expression for the language accepting all the string containing any number of a's and b's.

Solution:

The regular expression will be:

$$\text{r.e.} = (a + b)^*$$

This will give the set as $L = \{\epsilon, a, aa, b, bb, ab, ba, aba, bab, \dots\}$, any combination of a and b.

The $(a + b)^*$ shows any combination with a and b even a null string.

Examples of Regular Expression

Q4: Write the regular expression for the language accepting all the string which are starting with 1 and ending with 0, over $\Sigma = \{0, 1\}$.

Solution:

In a regular expression, the first symbol should be 1, and the last symbol should be 0. The r.e. is as follows:

$$R = 1 (0+1)^* 0$$

Q5: Write the regular expression for the language starting and ending with a and having any combination of b's in between.

Solution:

The regular expression will be:

$$R = a b^* a$$

Examples of Regular Expression

Q6:

Write the regular expression for the language starting with 'a' but not having consecutive b's.

Solution: The regular expression has to be built for the language:

$L = \{a, aba, aab, abaa, abab, \dots\}$

The regular expression for the above language is:

$R = \{a + ab\}^*$

Q7:

Write the regular expression for the language accepting all the string in which any number of a's is followed by any number of b's is followed by any number of c's.

Solution: As we know, any number of a's means a^* any number of b's means b^* , any number of c's means c^* . Since as given in problem statement, b's appear after a's and c's appear after b's. So the regular expression could be:

$R = a^* b^* c^*$

Examples of Regular Expression

Q8: Write the regular expression for the language over $\Sigma = \{0\}$ having even length of the string.

Solution:

The regular expression has to be built for the language:

$$L = \{\epsilon, 00, 0000, 000000, \dots\}$$

The regular expression for the above language is:

$$R = (00)^*$$

Q9:

Write the regular expression for the language having a string which should have at least one 0 and at least one 1.

Solution:

The regular expression will be:

$$R = [(0 + 1)^* 0 (0 + 1)^* 1 (0 + 1)^*] + [(0 + 1)^* 1 (0 + 1)^* 0 (0 + 1)^*]$$

Examples of Regular Expression

Q10: Describe the language denoted by following regular expression

r.e. = $(b^* (aaa)^* b^*)^*$

Solution:

The language can be predicted from the regular expression by finding the meaning of it.

We will first split the regular expression as:

r.e. = (any combination of b's) $(aaa)^*$ (any combination of b's)

$L = \{\text{The language consists of the string in which a's appear triples, there is no restriction on the number of b's}\}$

Q11:

Write the regular expression for the language L over $\Sigma = \{0, 1\}$ such that all the string do not contain the substring 01.

Solution:

The Language is as follows:

$L = \{\epsilon, 0, 1, 00, 11, 10, 100, \dots\}$

The regular expression for the above language is as follows:

$R = (1^* 0^*)$

Examples of Regular Expression

Q12:

Write the regular expression for the language containing the string over $\{0, 1\}$ in which there are at least two occurrences of 1's between any two occurrences of 1's between any two occurrences of 0's.

Solution: At least two 1's between two occurrences of 0's can be denoted by $(0111^*0)^*$. Similarly, if there is no occurrence of 0's, then any number of 1's are also allowed. Hence the r.e. for required language is:

$$R = (1 + (0111^*0))^*$$

Q13:

Write the regular expression for the language containing the string in which every 0 is immediately followed by 11.

Solution:

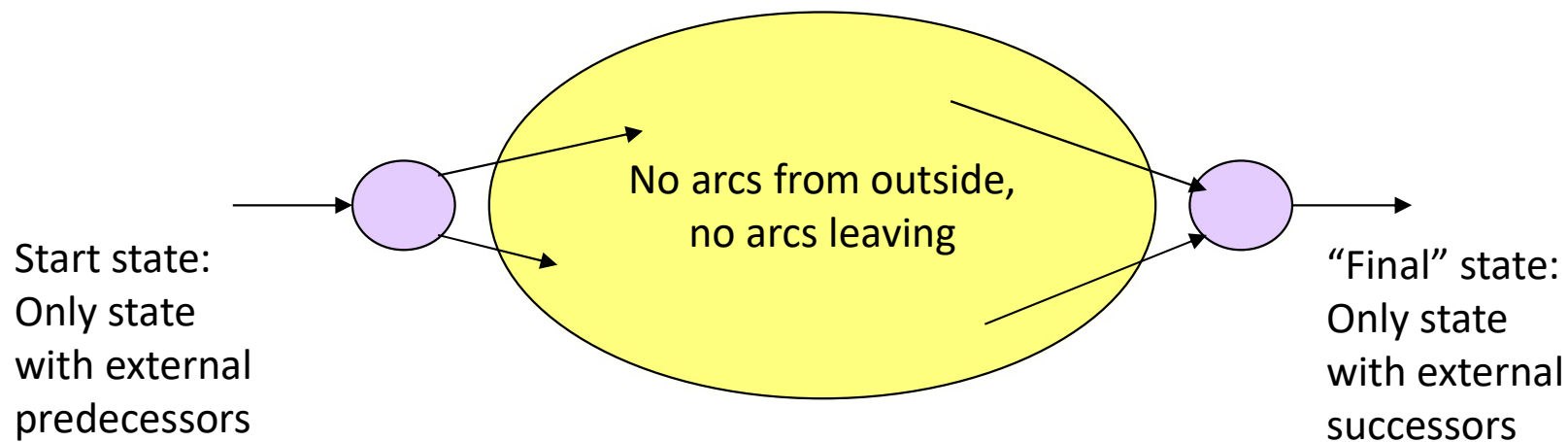
The regular expectation will be:

$$R = (011 + 1)^*$$

Converting a RE to an ϵ -NFA

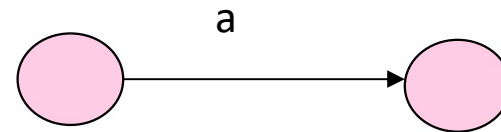
- Proof is an induction on the number of operators (+, concatenation, *) in the RE.
- We always construct an automaton of a special form (next slide).

Form of ϵ -NFA's Constructed

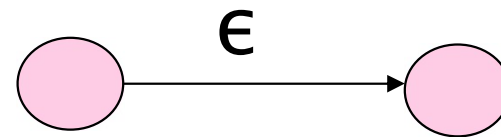


RE to ϵ -NFA: Basis

- Symbol **a**:



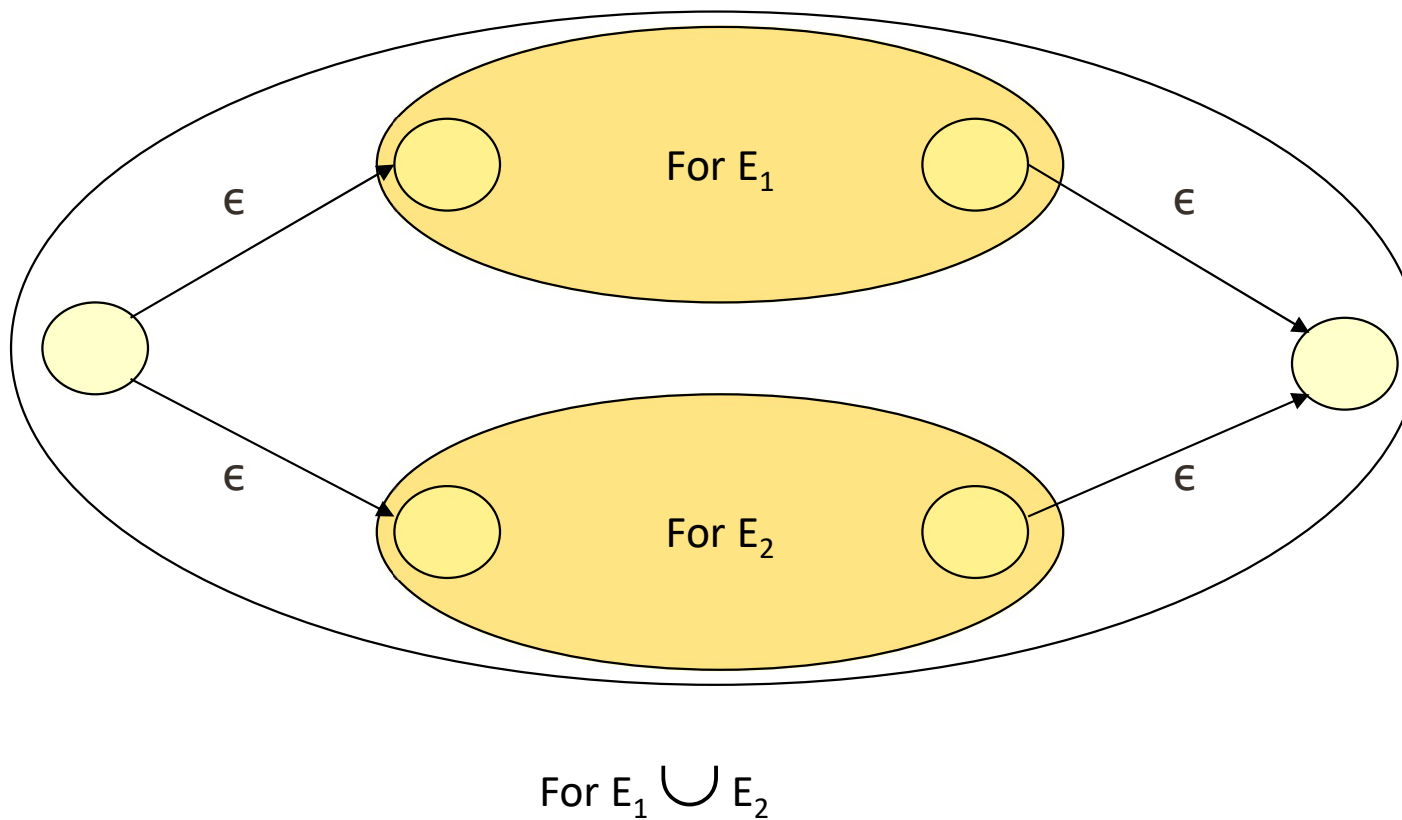
- ϵ :



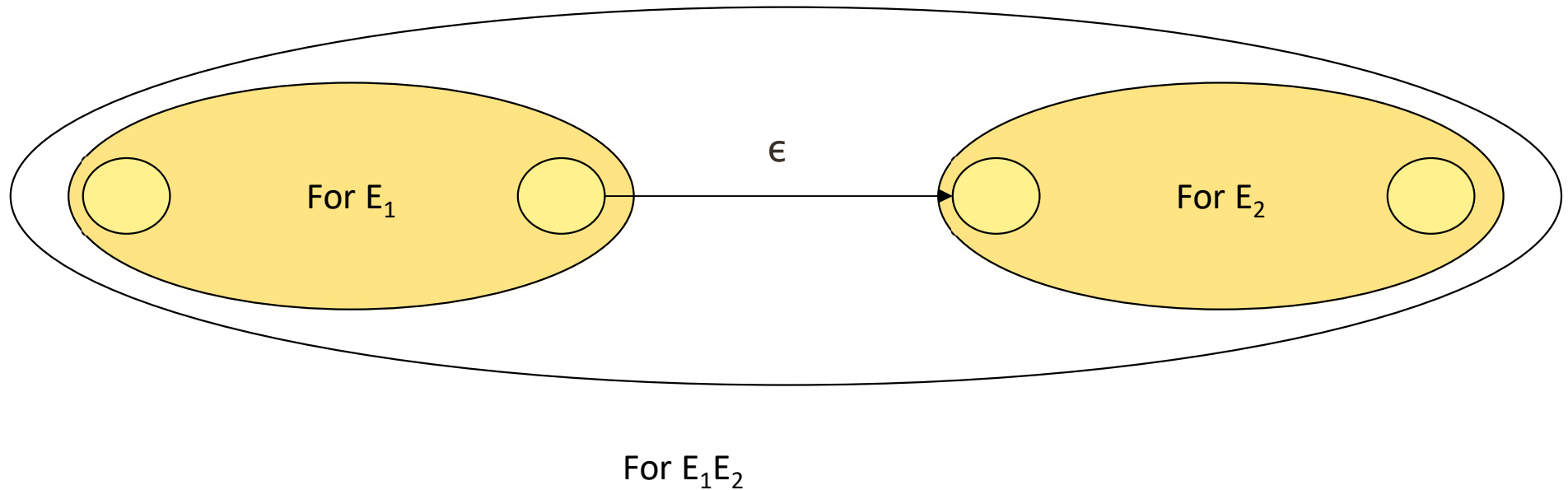
- $\emptyset$:



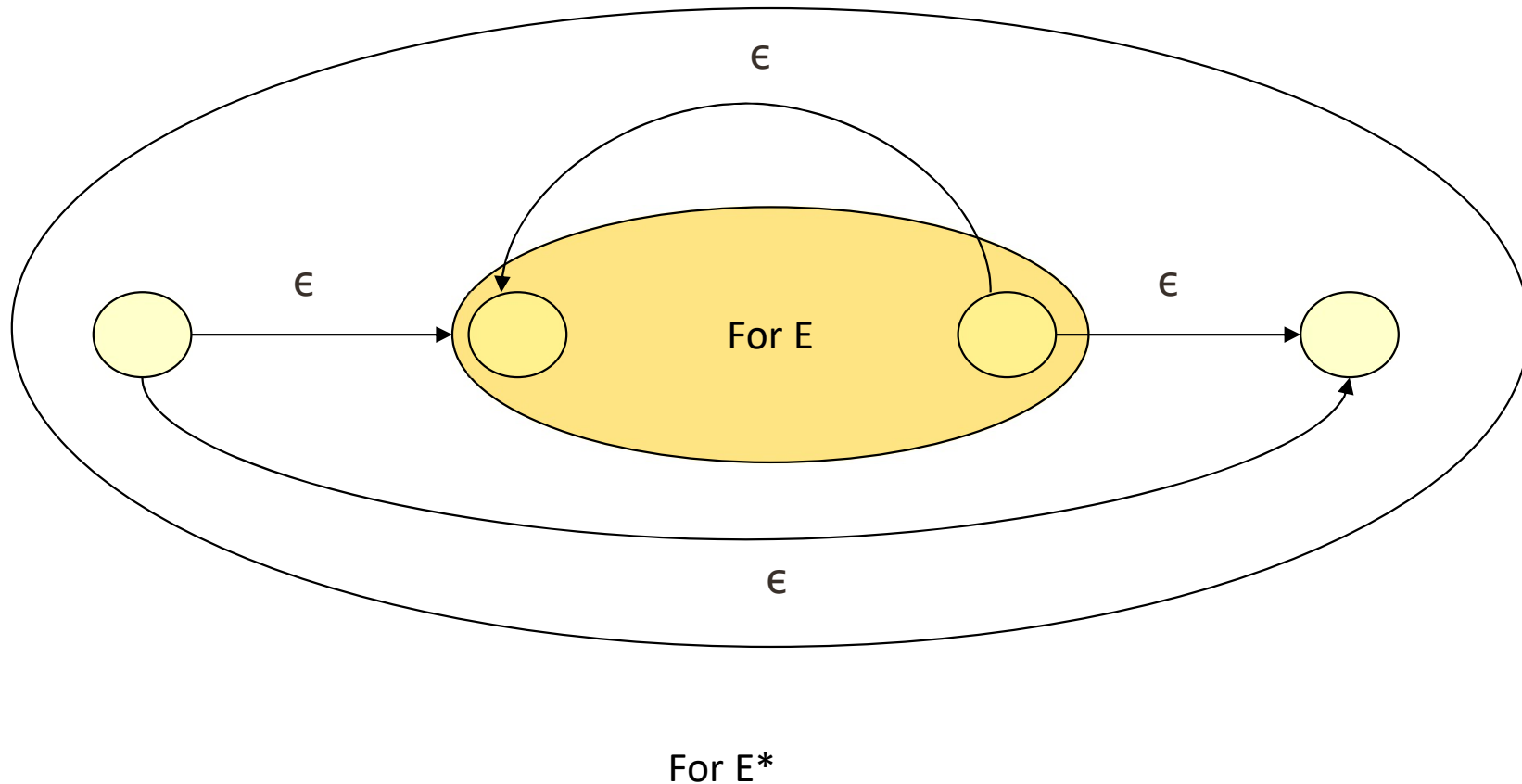
RE to ϵ -NFA: Induction 1 – Union



RE to ϵ -NFA: Induction 2 – Concatenation



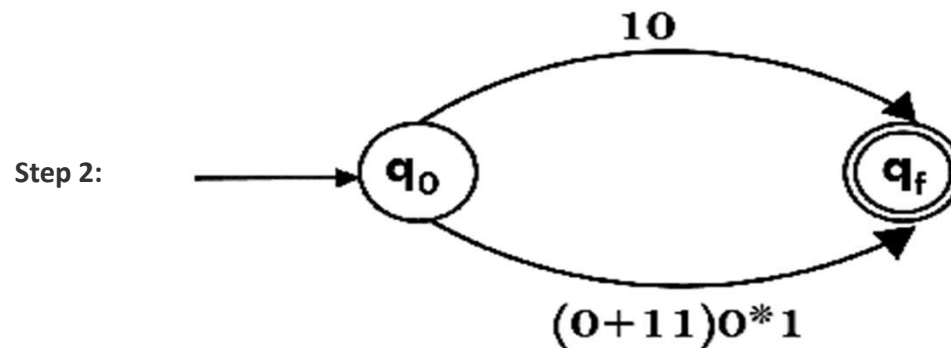
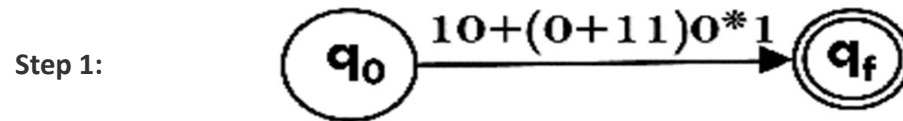
RE to ϵ -NFA: Induction 3 – Closure



Example of RE to NFA

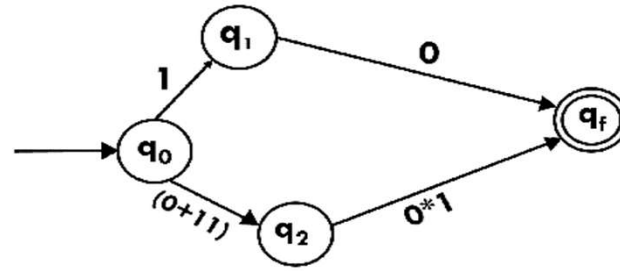
Design a FA from given regular expression $10 + (0 + 11)0^*1$.

Solution: First we will construct the transition diagram for a given regular expression.

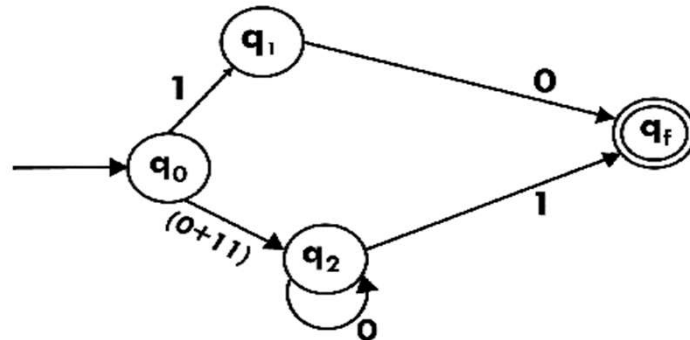


Example of RE to NFA

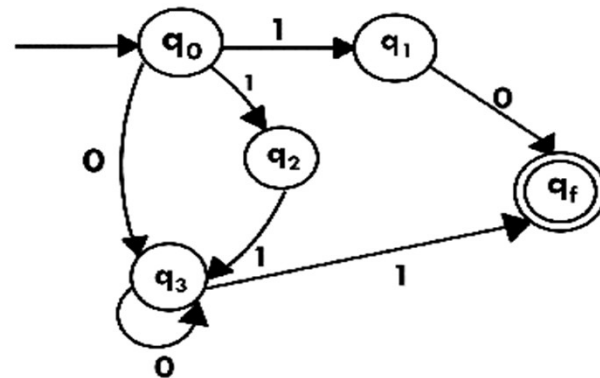
Step 3:



Step 4:



Step 5:

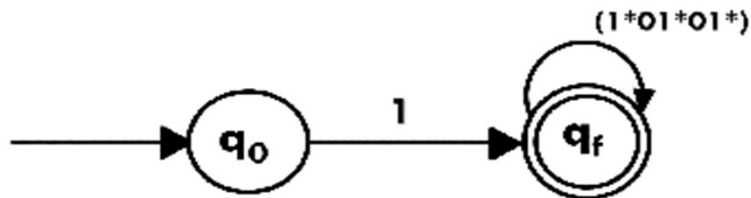
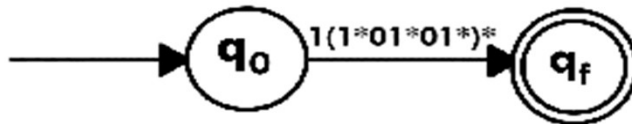


Example of RE to NFA

Design a NFA from given regular expression $1(1^*01^*01^*)^*$.

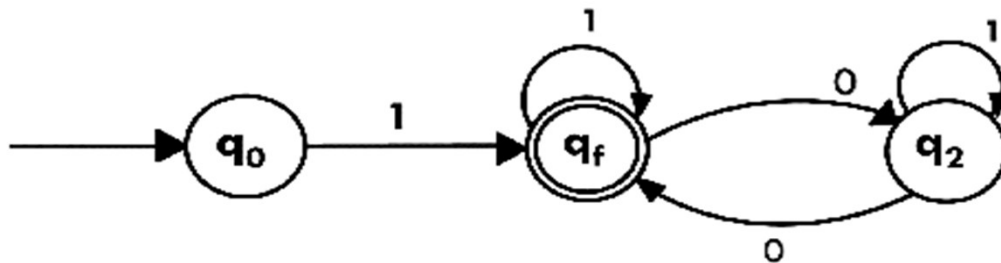
Solution: The NFA for the given regular expression is as follows:

Step 1:



Example of RE to NFA

Step 3:

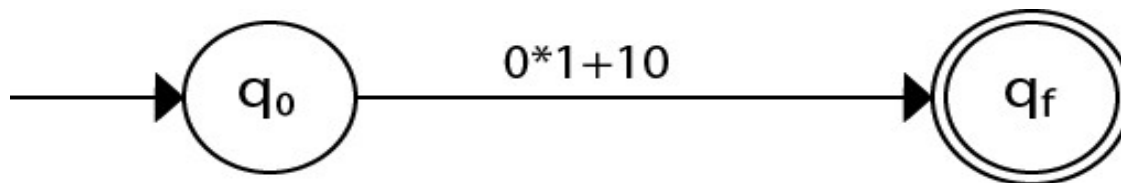


Construct the FA for regular expression $0^*1 + 10$.

Solution:

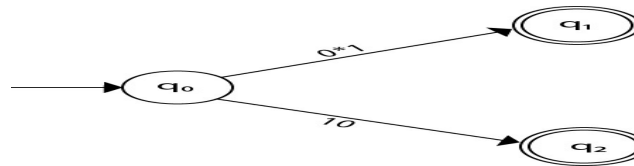
We will first construct FA for $R = 0^*1 + 10$ as follows:

Step 1:

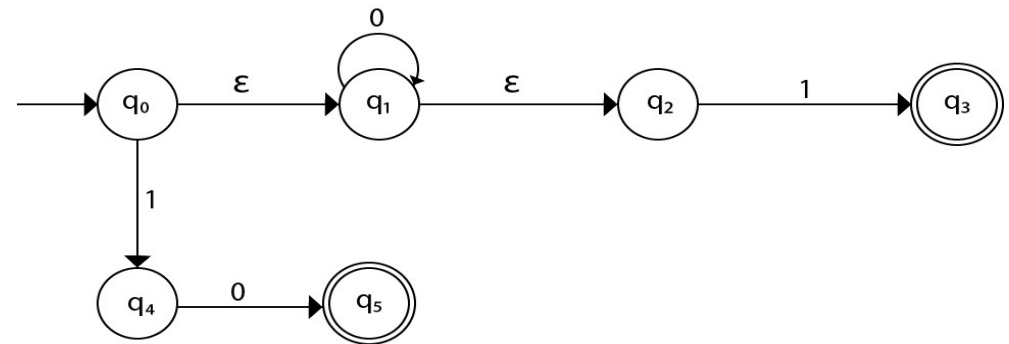


Example of RE to NFA

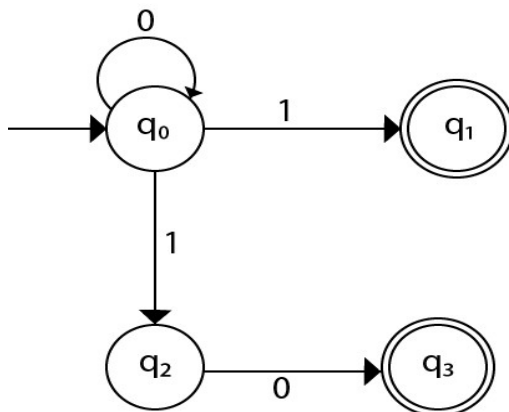
Step 2:



Step 3:

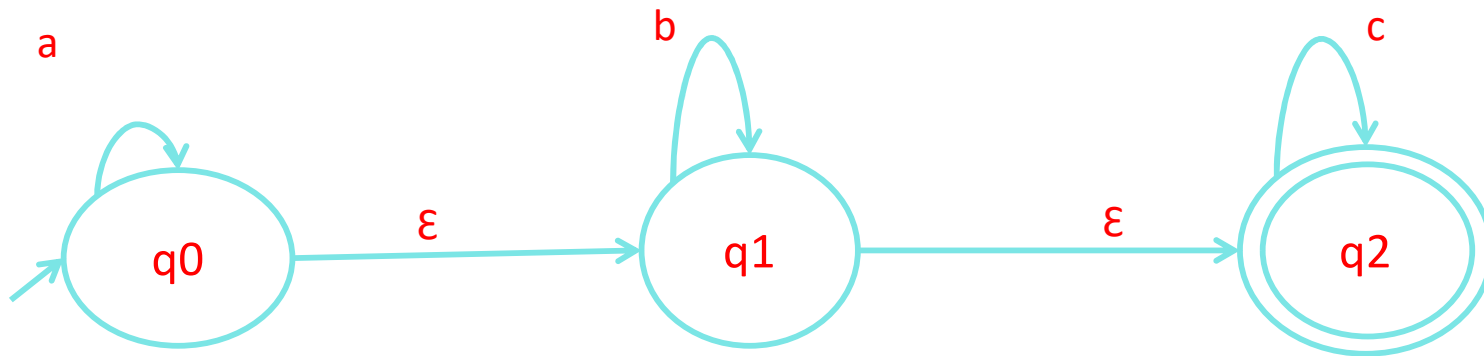


Step 4:



Regular Expression to NFA

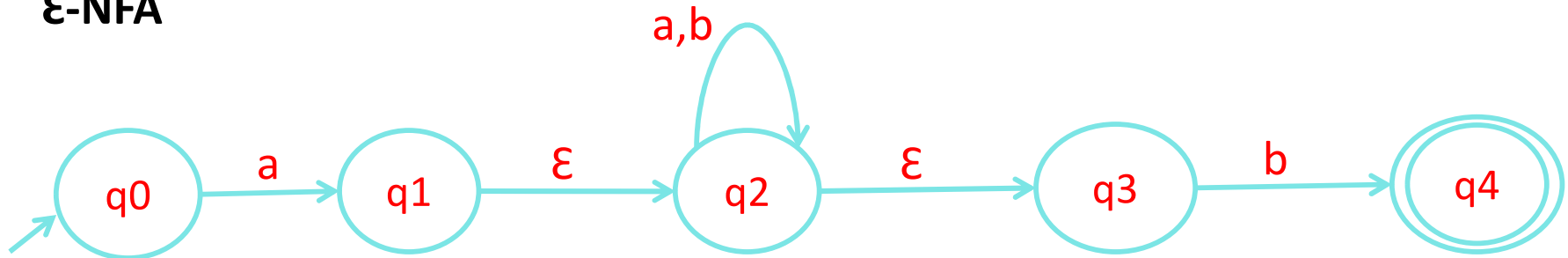
Construct an NFA for $a^*b^*c^*$.



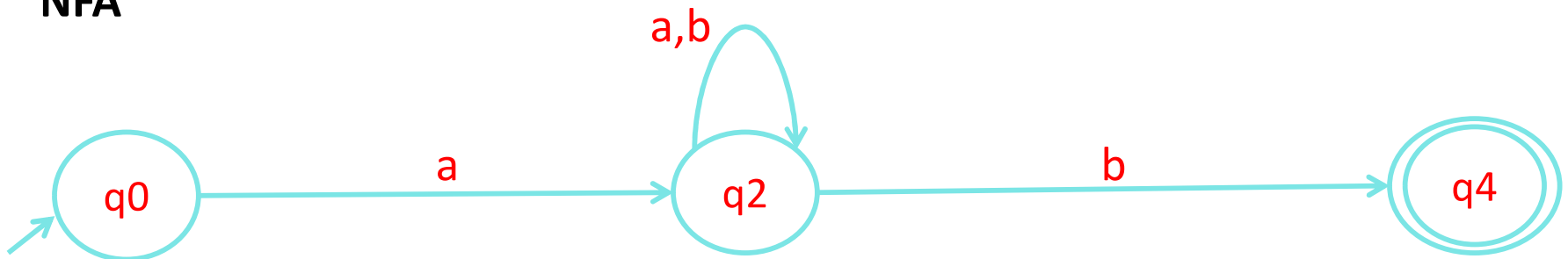
Regular Expression to NFA

Construct an NFA for $a(a+b)^*b$.

ϵ -NFA



NFA



Arden's Theorem

Let **P** and **Q** be two regular expressions. If P does not contain ϵ , then

R = Q + RP has a unique solution that is

$$R = QP^*$$

Proof

$$R = Q + RP$$

$$= Q + QP^*P$$

$$= Q(\epsilon + P^*P)$$

$$= QP^*$$

So, $R = QP^*$ is a solution.

$$R = Q + RP$$

$$= Q + (Q + RP)P$$

$$= Q + QP + RP^2$$

$$= Q + QP + (Q + RP)PP$$

$$= Q + QP + QP^2 + RP^3$$

$$= \dots$$

$$= \dots$$

$$= Q + QP + QP^2 + \dots + QP^n + RP^{n+1}$$

$$= Q + QP + QP^2 + \dots + QP^n + QP^*P^{n+1}$$

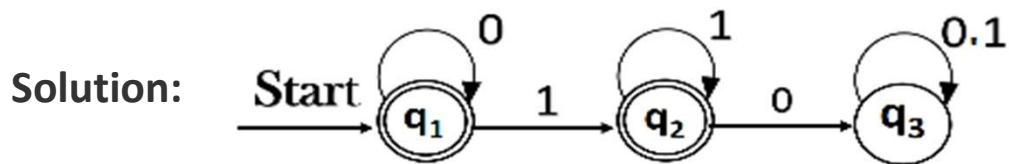
$$= Q(\epsilon + P + P^2 + \dots + P^n + P^*P^{n+1})$$

$$= QP^*$$

Hence, $R = QP^*$ is a Unique Solution.

Example of Arden's theorem

Q1: Construct the regular expression for the given DFA



Let us write down the equations

$$q_1 = q_1 0 + \epsilon$$

Since q_1 is the start state, so ϵ will be added, and the input 0 is coming to q_1 from q_1 hence we write
 State = source state of input $\times$ input coming to it

Similarly,

$$q_2 = q_1 1 + q_2 1$$

$$q_3 = q_2 0 + q_3 (0+1)$$

Example of Arden's theorem

Since the final states are q_1 and q_2 , we are interested in solving q_1 and q_2 only. Let us see q_1 first

$$q_1 = q_1 0 + \epsilon$$

We can re-write it as

$$q_1 = \epsilon + q_1 0$$

Which is similar to $R = Q + RP$, and gets reduced to $R = QP^*$.

Assuming $R = q_1$, $Q = \epsilon$, $P = 0$

We get

$$q_1 = \epsilon.(0)^* \quad q_1 = 0^* \quad (\epsilon.R^* = R^*)$$

Substituting the value into q_2 , we will get

$$q_2 = 0^* 1 + q_2 1 \quad q_2 = 0^* 1 (1)^* \quad (R = Q + RP \rightarrow QP^*)$$

The regular expression is given by

$$r = q_1 + q_2 = 0^* + 0^* 1.1^* \quad r = 0^* + 0^* 1^+ \quad (1.1^* = 1^+)$$

Example of Arden's theorem

Q2: Find regular expression for the following DFA using Arden's Theorem-

Solution-

Step-01:

Form an equation for each state-

$$q_1 = \epsilon \dots\dots(1)$$

$$q_2 = q_1.a \dots\dots(2)$$

$$q_3 = q_1.b + q_2.a + q_3.a \dots\dots(3)$$

Step-02:

Bring final state in the form $R = Q + RP$.

Using (1) in (2), we get-

$$q_2 = \epsilon.a$$

$$q_2 = a \dots\dots(4)$$

Example of Arden's theorem

Using (1) and (4) in (3), we get-

$$q3 = q1.b + q2.a + q3.a$$

$$q3 = \epsilon.b + a.a + q3.a$$

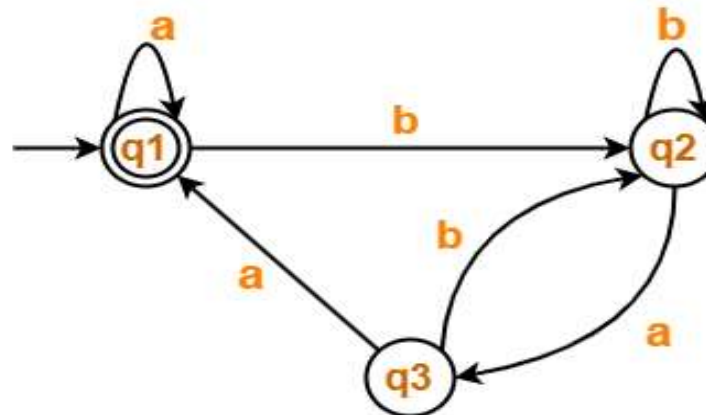
$$q3 = (b + a.a) + q3.a \dots\dots(5)$$

Using Arden's Theorem in (5), we get-

$$q3 = (b + a.a)a^*$$

Thus, Regular Expression for the given DFA = $(b + aa)a^*$

Q3: Find regular expression for the following DFA using Arden's Theorem-



Vivek Kumar Sharma BCSE0404(TAFL) (Unit II)

Example of Arden's theorem

Step-01:

Form an equation for each state-

$$q1 = \epsilon + q1.a + q3.a \dots\dots(1)$$

$$q2 = q1.b + q2.b + q3.b \dots\dots(2)$$

$$q3 = q2.a \dots\dots(3)$$

Step-02:

Bring final state in the form $R = Q + RP$.

Using (3) in (2), we get-

$$q2 = q1.b + q2.b + q2.a.b$$

$$q2 = q1.b + q2.(b + a.b) \dots\dots(4)$$

Using (3) in (2), we get-

$$q2 = q1.b + q2.b + q2.a.b$$

$$q2 = q1.b + q2.(b + a.b) \dots\dots(4)$$

Example of Arden's theorem

Using (5) in (3), we get-

$$q_3 = q_1.b.(b + a.b)^*.a \dots\dots(6)$$

Using (6) in (1), we get-

$$q_1 = \epsilon + q_1.a + q_1.b.(b + a.b)^*.a.a$$

$$q_1 = \epsilon + q_1.(a + b.(b + a.b)^*.a.a) \dots\dots(7)$$

Using Arden's Theorem in (7), we get-

$$q_1 = \epsilon.(a + b.(b + a.b)^*.a.a)^*$$

$$q_1 = (a + b.(b + a.b)^*.a.a)^*$$

Thus, Regular Expression for the given DFA = $(a + b(b + ab)^*aa)^*$

Algebraic method using Arden's theorem

Arden's theorem state that: "If P and Q are two regular expressions over " Σ ", and if P does not contain " ϵ ", then the following equation in R given by $R = Q + RP$ has a unique solution i.e., $R = QP^*$." That means, whenever we get any equation in the form of $R = Q + RP$, then we can directly replace it with $R = QP^*$. So, here we will first prove that $R = QP^*$ is the solution of this equation and then prove that it is the unique solution of this equation. Let's start by taking this equation as equation (i)

1. proof $R = QP^*$ is the solution of $R = Q + RP$

$R = Q + RP$ (i) Now, replacing R by $R = QP^*$, we get,

$R = Q + QP^*P$ Taking Q as common,

$$R = Q(\epsilon + P^*P) = QP^*$$

(As we know that $\epsilon + R^*R = R^*$). **Hence proved. Thus, $R = QP^*$ is the solution of the equation $R = Q + RP$. Now, we have to prove that this is the only solution to this equation.**

Algebraic method using Arden's theorem

2. proof $R = QP^*$ is the unique solution of $R = Q + RP$

Let me take this equation again:

$R = Q + RP$ Now, replace R by $R = Q + RP$,

$$R = Q + (Q + RP)P = Q + QP + RP^2$$

Again, replace R by $R = Q + RP$

$$R = Q + QP + (Q + RP)P^2 = Q + QP + QP^2 + RP^3 \dots = Q + QP + QP^2 + \dots + Q + RP^{(n+1)}$$

Now, replace R by $R = QP^*$, we get,

$$R = Q + QP + QP^2 + \dots + QP^n + QP^*P^{(n+1)}$$

Taking Q as common,

$$R = Q(\epsilon + P + P^2 + \dots + P^n + P^*P^{(n+1)}) = QP^*$$

[As $\epsilon + P + P^2 + \dots + P^n + P^*P^{(n+1)}$ represent the closure of P] Hence proved. **Thus, $R = QP^*$ is the unique solution of the equation $R = Q + RP$.**

Identities of Regular Expression

- I₁:** $R + \emptyset = \emptyset + R = R$
- I₂:** $R \cdot \emptyset = \emptyset$
- I₃:** $R \cdot \varepsilon = \varepsilon \cdot R = R$
- I₄:** $\varepsilon^* = \varepsilon$
- I₅:** $R + R = R$
- I₆:** $R^* R^* = R^*$
- I₇:** $RR^* = R^*R$
- I₈:** $(R^*)^* = R^*$
- I₉:** $\varepsilon + RR^* = \varepsilon + R^*R = R^*$
- I₁₀:** $(PQ)^*P = P(QP)^*$
- I₁₁:** $(P+Q)^* = (P^*Q^*)^* = (P^*+Q^*)^*$
- I₁₂:** $PQ+PR = P(Q+R) \text{ and } QP+RP = (Q+R)P$

Examples of Regular Expression

$$L(\mathbf{01}) = \{01\}.$$

$$L(\mathbf{01+0}) = \{01, 0\}.$$

$$L(\mathbf{0(1+0)}) = \{01, 00\}.$$

Note order of precedence of operators.

$$L(\mathbf{0^*}) = \{\epsilon, 0, 00, 000, \dots\}.$$

$$L(\mathbf{(0+10)^*(\epsilon+1)}) = \text{all strings of 0's and 1's without two consecutive 1's.}$$

Regular grammar

Type-3 grammar/Regular grammar:

Regular grammar generates regular language. They have a single non-terminal on the left-hand side and a right-hand side consisting of terminal's or terminal's followed by a non-terminal or vice versa.

The productions must be in the form:

$$A \rightarrow wB$$

$$A \rightarrow w$$

Where $w \in T^*$

$A \rightarrow Bw$ where $A, B \in \text{Variable}(V)$ and $w \in T^*$ i.e. string of terminals.

Types of regular grammar:

1. Left Linear grammar(LLG)
2. Right linear grammar(RLG)

Regular grammar

1. Left linear grammar(LLG):

In LLG, the productions are in the form if all the productions are of the form

$$A \rightarrow Bw$$

$$A \rightarrow w \text{ where } A, B \in V \text{ and } w \in T^*$$

2. Right linear grammar(RLG):

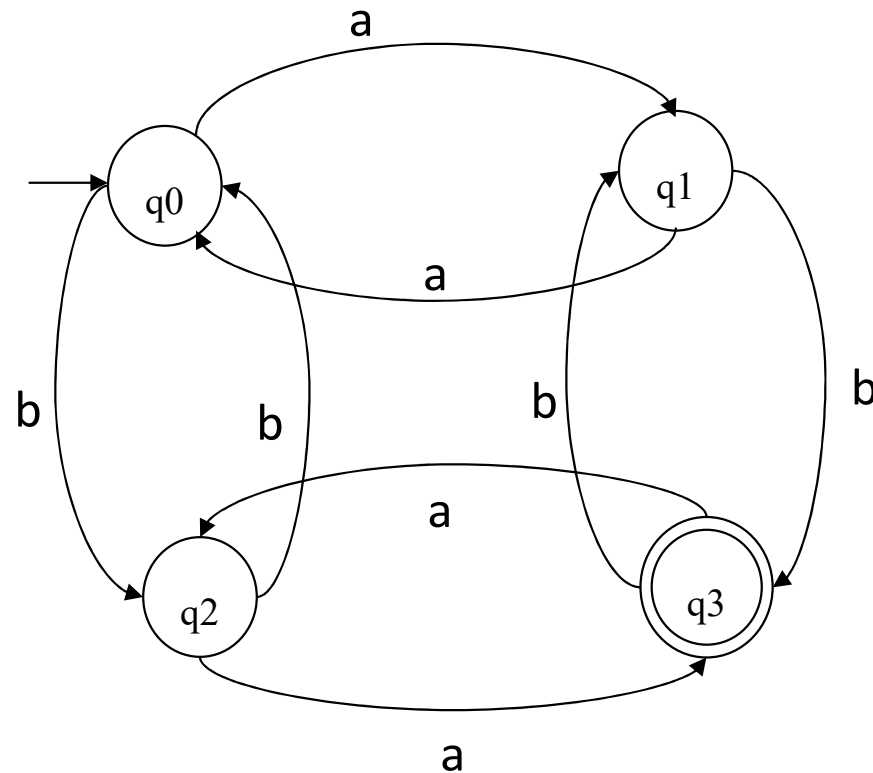
In RLG, the productions are in the form if all the productions are of the form

$$A \rightarrow wB$$

$$A \rightarrow w \text{ where } A, B \in V \text{ and } w \in T^*$$

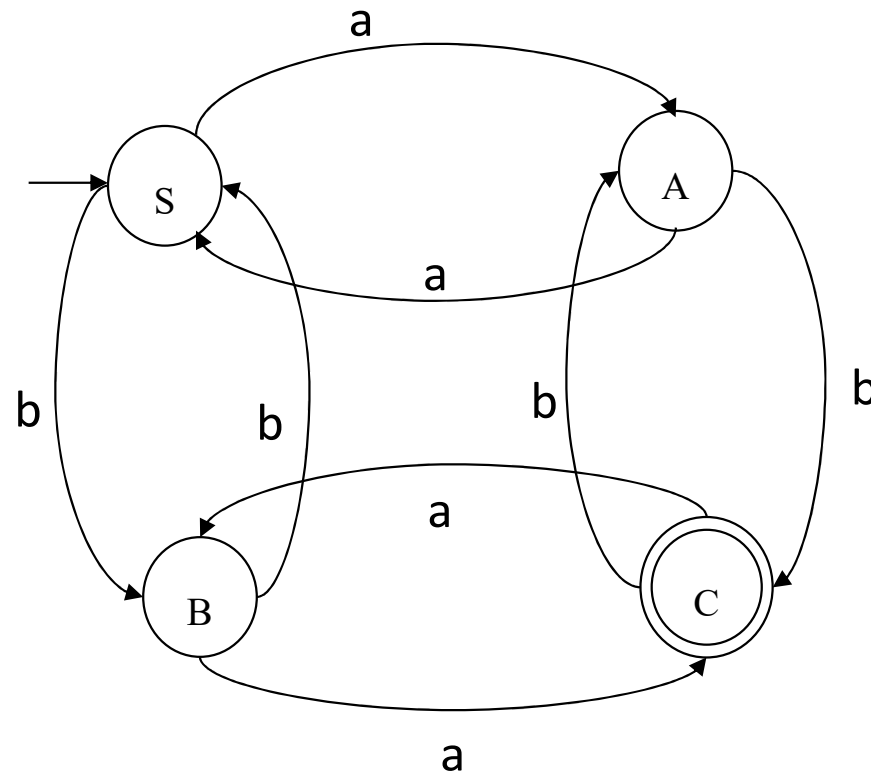
DFA to Right Linear Regular Grammar

- Given a DFA:



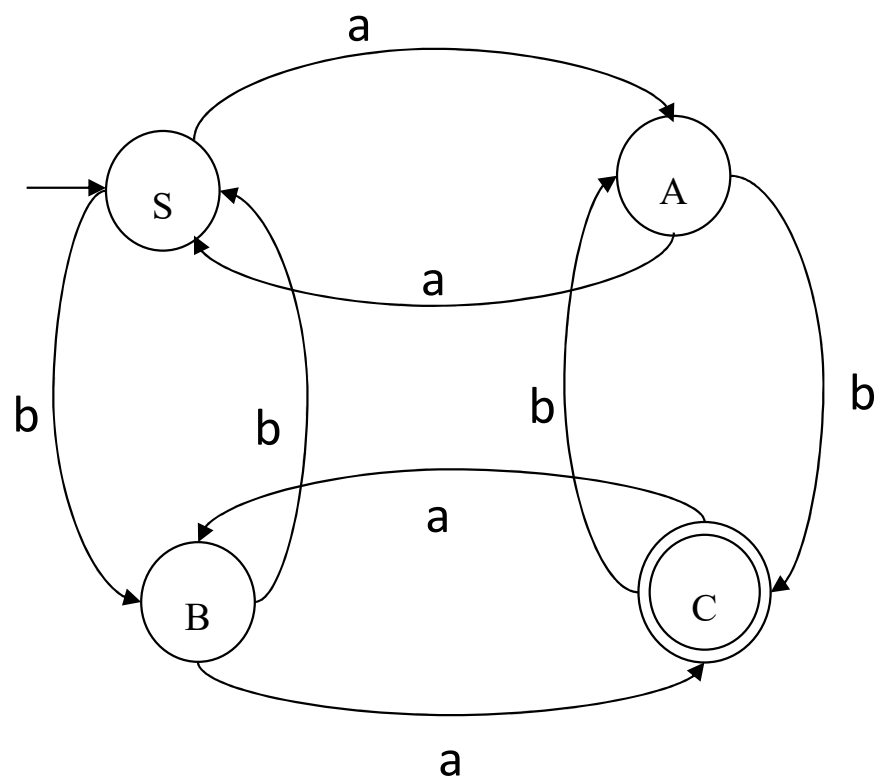
DFA to Right Linear Regular Grammar (Continued)

- **Step 1:** Rename the states: for q_0 which is start state make it as S . rest as non terminals like $A, B, C, \dots$



DFA to Right Linear Regular Grammar (Continued)

- Step 2: Set of productions are given by:



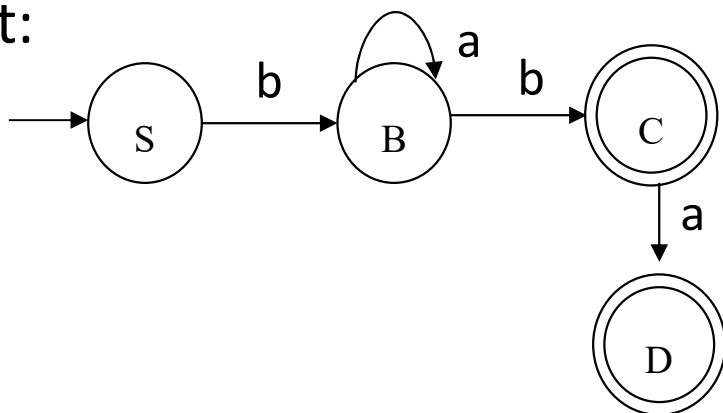
- $S \rightarrow aA \mid bB$
- $A \rightarrow aS \mid bC \mid b$
- $B \rightarrow bs \mid aC \mid a$
- $C \rightarrow aB \mid bA$

Right Linear Grammar to DFA

Convert the given Right Linear Grammar to DFA :

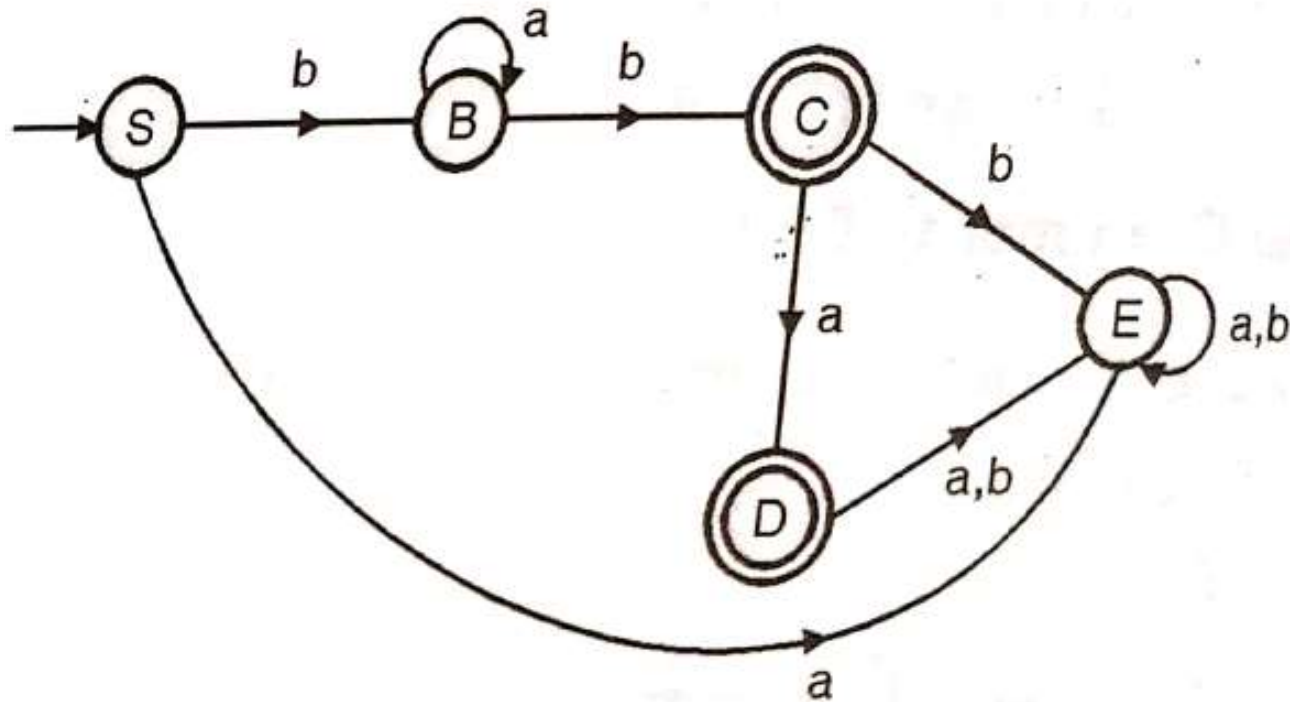
- $S \rightarrow bB$
- $B \rightarrow bC \mid b$
- $B \rightarrow aB$
- $C \rightarrow a$

STEP 1: Adding transactions corresponding to every production ,we get:



Right Linear Grammar to DFA (Continued)

- **STEP 2:** Adding a state E to handle Φ transitions, final DFA will be:



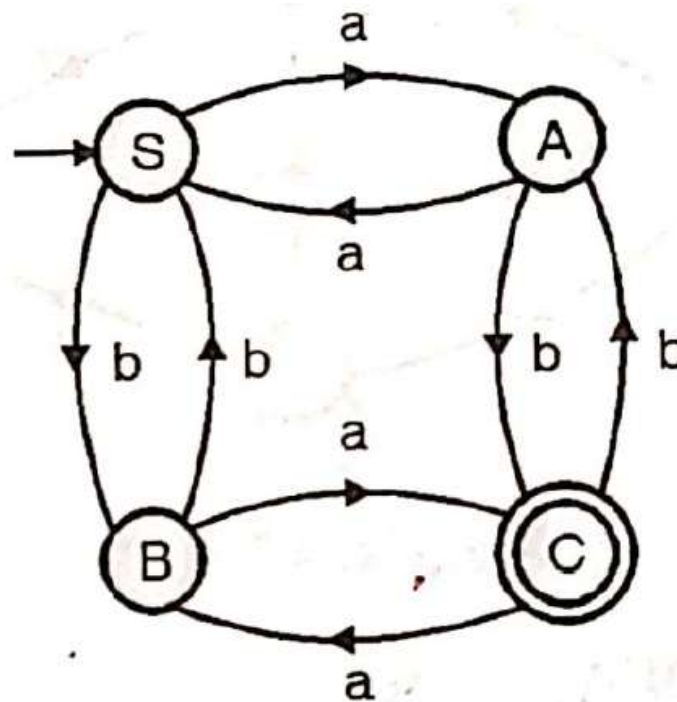
DFA to Left Linear Grammar

Following steps are required to write a left linear grammar corresponding to a DFA.

- Interchange starting state and the final state.
- Reverse the direction of all the transactions.
- Write the grammar from the transaction graph in left linear form.

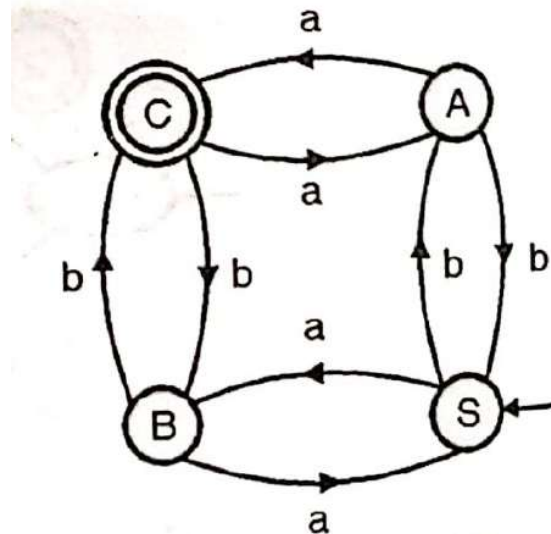
DFA to Left Linear Grammar

- Following DFA is Given:



DFA to Left Linear Grammar(Continued)

- Interchange the starting state and the final state and reverse the direction of transactions, we get the transaction graph as following:



- Writing an equivalent left linear grammar, we get:
- $S \rightarrow Ba \mid Ab, A \rightarrow Sb \mid Ca \mid a$
- $B \rightarrow Sa \mid Cb \mid b, C \rightarrow Bb \mid Aa$

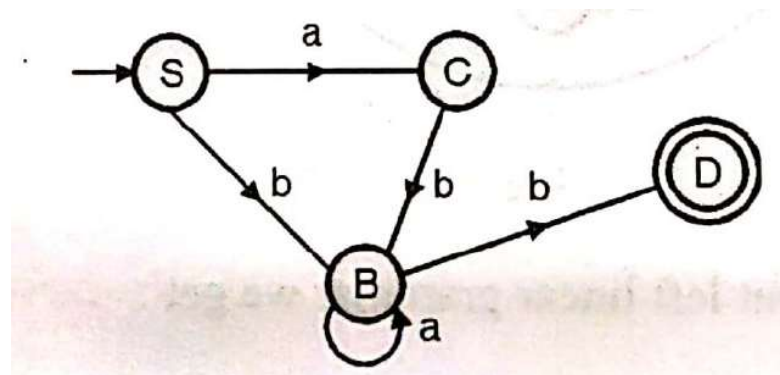
Left Linear Grammar to DFA

Every left linear grammar can be represented using an equivalent DFA. Following steps are required to draw a DFA for a given left linear grammar.

- Draw a transaction graph from the given left linear grammar.
- Reverse the direction of all the transactions.
- Interchange starting state and the final state.
- Carry out conversion from FA to DFA.

Left Linear Grammar to DFA (Example)

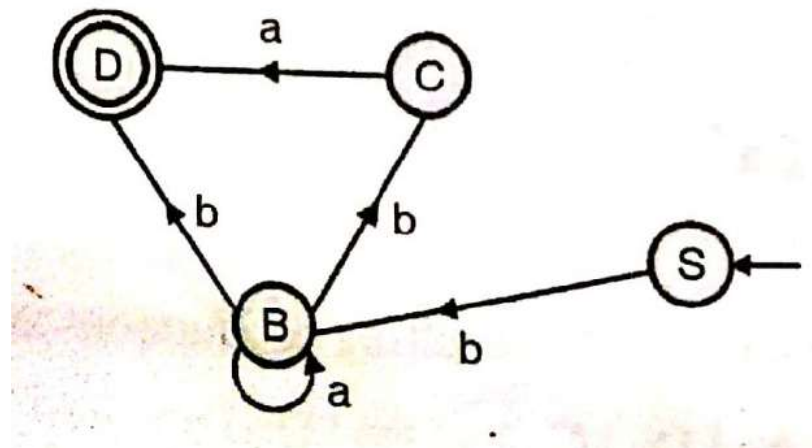
- Following Linear grammar is given:
- $S \rightarrow Ca \mid Bb$
- $C \rightarrow Bb$
- $B \rightarrow Ba \mid b$
- Step 1: Draw a transaction graph from the given left linear grammar:



- D is an accepting state. It is added for the production $B \rightarrow b$

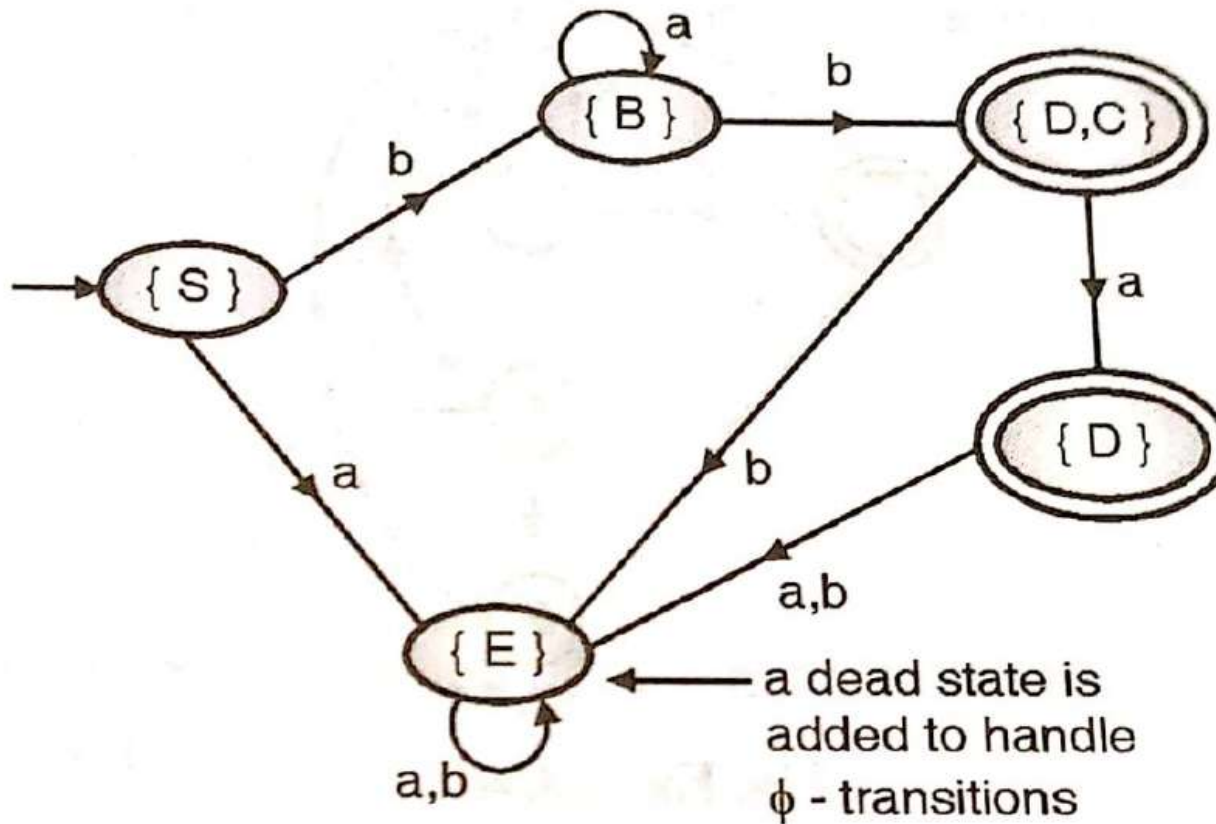
Left Linear Grammar to DFA (Example)

- **Step 2:** Reverse the direction of transaction and interchange starting state and the final state.



Left Linear Grammar to DFA (Example)

- Conversion from FA to DFA:



Video Links

My Video Channel:

<https://www.youtube.com/playlist?list=PLKhyqvN-JcP5AF4UzRz9wxFVBm63igm7y>

- NPTEL Video Links

<https://youtu.be/vQeROGcwQs4>

https://youtu.be/XxuH_K-wzpg

<https://youtu.be/C7LqbEunJ1Y>

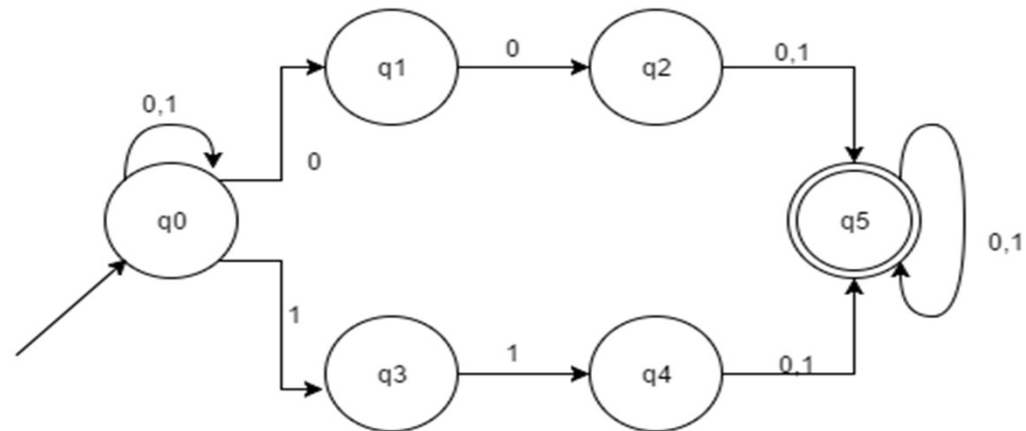
Daily Quiz

1. If L_1 and L_2 are regular languages is/are also regular language(s).
 - a) $L_1 + L_2$
 - b) L_1L_2
 - c) L_1'
 - d) All of above

2. Every regular expression can be expressed as CFG but every CFG cannot be expressed as a regular expression. this statement is :
 - a) True
 - b) False
 - c) depends on language
 - d) None of the option

Daily Quiz

3. The given NFA corresponds to which of the following Regular expressions?



- A. $(0+1)^*(00+11)(0+1)^+$
- B. $(0+1)^*(00+11)^*(0+1)^*$
- C. $(0+1)^*(00+11)(0+1)$
- D. $(0+1)(00+11)(0+1)^*$

4. Regular grammar is
 - a) context free grammar
 - b) non context free grammar
 - c) English grammar
 - d) none of the mentioned
5. Which of the following is true?
 - a) Every subset of a regular set is regular
 - b) Every finite subset of non-regular set is regular
 - c) The union of two non regular set is not regular
 - d) Infinite union of finite set is regular
6. Regular expressions are closed under
 - a) Union
 - b) Intersection
 - c) Kleen star
 - d) All of the mentioned

Weekly Assignment

1. Find a regular expression corresponding to each of the following subsets of $\{a, b\}$. **[CO2]**

- The set of all strings containing exactly 2a's.
- The set of all strings containing at least 2a's.
- The set of all strings containing at most 2a's.
- The set of all strings containing the substring aa.

2. Find the sets represented by the following regular expressions. **[CO2]**

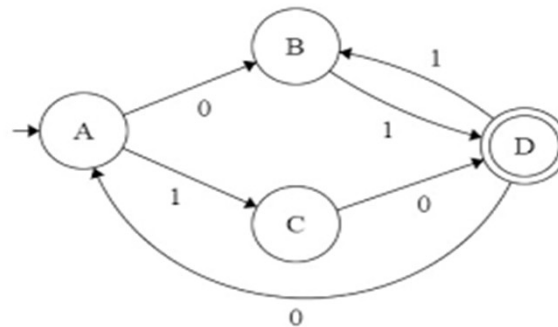
- $(a + b)^*(aa + bb + ab + ba)^*$
- $(aa)^* + (aaa)^*$
- $(1 + 01 + 001)^*(\epsilon + 0 + 00)$
- $a + b(a + b)^*$

Weekly Assignment

3. Prove the identity: $(a^*ab + ba)^*a^* = (a + ab + ba)^*$

[CO2]

4. Find the regular expression corresponding to the automaton given in Fig.



Weekly Assignment

5. State and prove Arden's theorem. Give a regular expression for the Finite Automaton $M = [\{A, B, C, D\}, \{a, b\}, \delta, A, \{A\}]$ where δ is given below:

[CO2]

– $\delta(A, a) = B$

– $\delta(B, a) = D$

– $\delta(C, a) = A$

– $\delta(D, a) = D$

$\delta(A, b) = C$

$\delta(B, b) = A$

$\delta(C, b) = D$

$\delta(D, b) = D$

MCQ s

Let a Grammar G is defined by the production $S \rightarrow aS \mid bS$, then language generated by Grammar is defined as

- A. Φ
- B. ϵ
- C. $\{a,b\}^*$
- D. $\{a,b\}^+$

Which of the following is true?

- A. $(01)^*0 = 00(10)^*$
- B. $(0+1)^*0(0+1)^*1(0+1) = (0+1)^*01(0+1)^*$
- C. $(0+1)^*01(00+1)^*+1*0^* = (0+1)^*$
- D. All of the mentioned

MCQ s

Which among the following looks similar to the given expression? $((0+1)^*(0+1))^*$

- A. $\{x \in \{0,1\}^* \mid x \text{ is all binary number with even length}\}$
- B. $\{x \in \{0,1\} \mid x \text{ is all binary number with even length}\}$
- C. $\{x \in \{0,1\}^* \mid x \text{ is all binary number with odd length}\}$
- D. $\{x \in \{0,1\} \mid x \text{ is all binary number with odd length}\}$

Which of the following represents a language which has no pair of consecutive 1's if $\Sigma = \{0,1\}$?

- A. $(0+10)^*(1+\epsilon)$
- B. $(0+10)^*(1+\epsilon)^*$
- C. $(0+101)^*(0+\epsilon)$
- D. $(1+010)^*(1+\epsilon)$

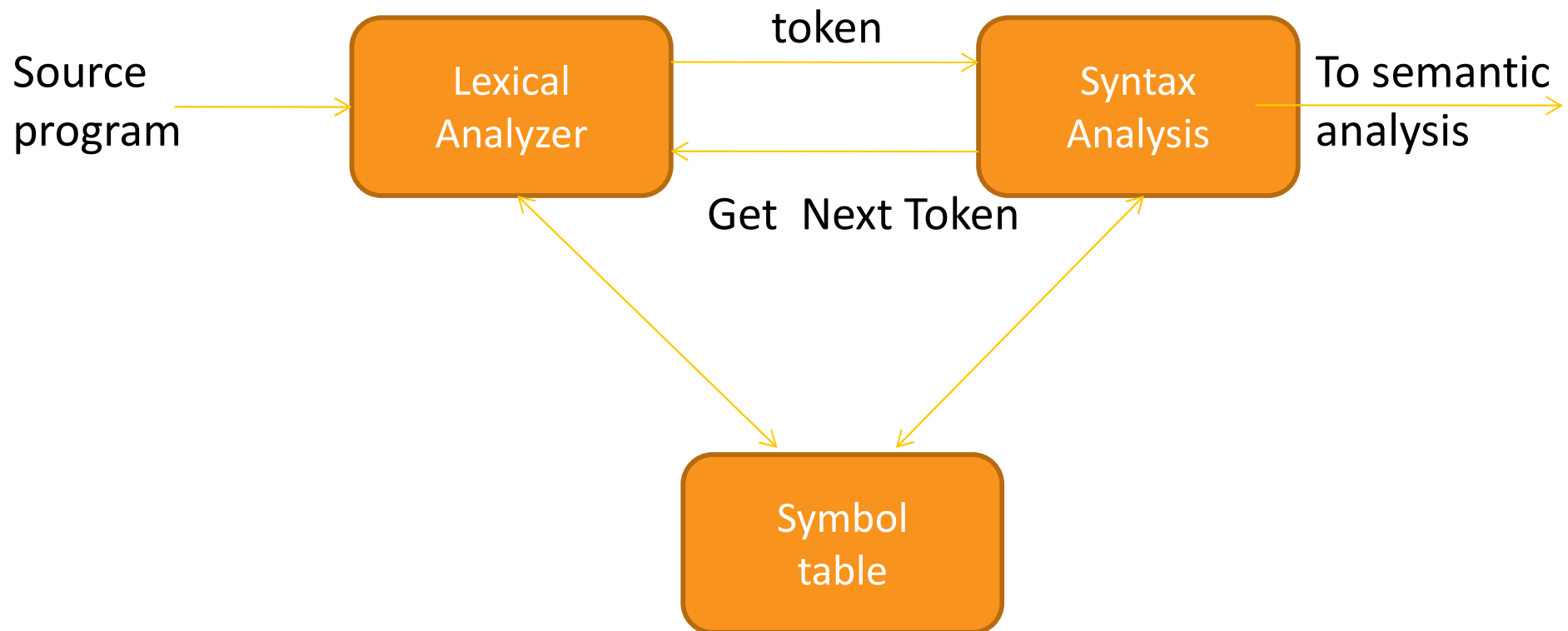
Topic Objective(CO1)

1. Introduce students to the concepts of scanner how a lexical analyzer work in compiler designing

Lexical Analyszer

- Lexical Analyzer coverts Source Program into Stream of Tokens.
- Also called Scanning Phase.
- Lexical Analyzer's main task is to read the input characters and produce output as sequence of tokens that the parser uses for syntax analysis.

- Interface diagram of Lexical Analysis



Functions of Lexical Analyzer(CO1)

- Reads the input program character by character & produce a stream of token.
- Removing white spaces / tabs.
- Remove comments from source program.
- Lexical Analyzer generates errors and gives the line number of the error.

- **Tokens**- Sequence of characters that have a collective meaning and treated as a single logical entity.
- Tokens are symbolic names for the entities that make up the text of the program.
- Typical Tokens are:

{ **Keywords** , **Identifiers** , **Operators** , **Separators** etc }

(for, if, while) **(variable name,** **(+, - , * , ++)** **(, ; etc)**
function name)

Ex: **int** **a** **=** **15** **;**

- (keyword) (identifier) (operator) (constant) (separator)

Keyword: Word (for, if, else, ...) which may have a particular predefined meaning to the compiler.

Identifier : Words that the programmer construct to attach a name .

Operator: Symbols used for arithmetic , character or logical operations such as +, - , = , !

Numeric Constant : Numbers such as 124, 12.35 , 0.09 E – 23 etc. These must be converted to a numeric format so that they can be used in arithmetic operations.

Character Constant : Single character or string of characters enclosed in quotes.

Special Characters : Character used as delimiters such as . , (,) , { , } , ; . These are generally single character words.

Note :

Comments : Not put out as tokens to next phase of compiler.

White Space : Space and tabs are generally ignored by the compiler and are not put out as tokens.

New lines: New line characters should also be ignored.

Count the number of tokens.

```
int main ( )  
  
{  
    /* find the Max of a and b */  
    int a = 20 ,    b  = 30  ;  
    if ( a < b )  
        return ( b ) ;  
    else  
        return ( a )  ;  
}
```

Count the number of tokens.

```
1  2  3  4
int main ( )
5
{
    /* find the Max of a and b */
6  7  8  9 10 11 12 13 14
int a = 20 ,   b  = 30 ;
15 16 17 18 19 20
    if ( a < b )
        21  22 23 24 25
        return ( b ) ;
26
    else
        27  28 29 30 31
        return ( a ) ;
32
}
```

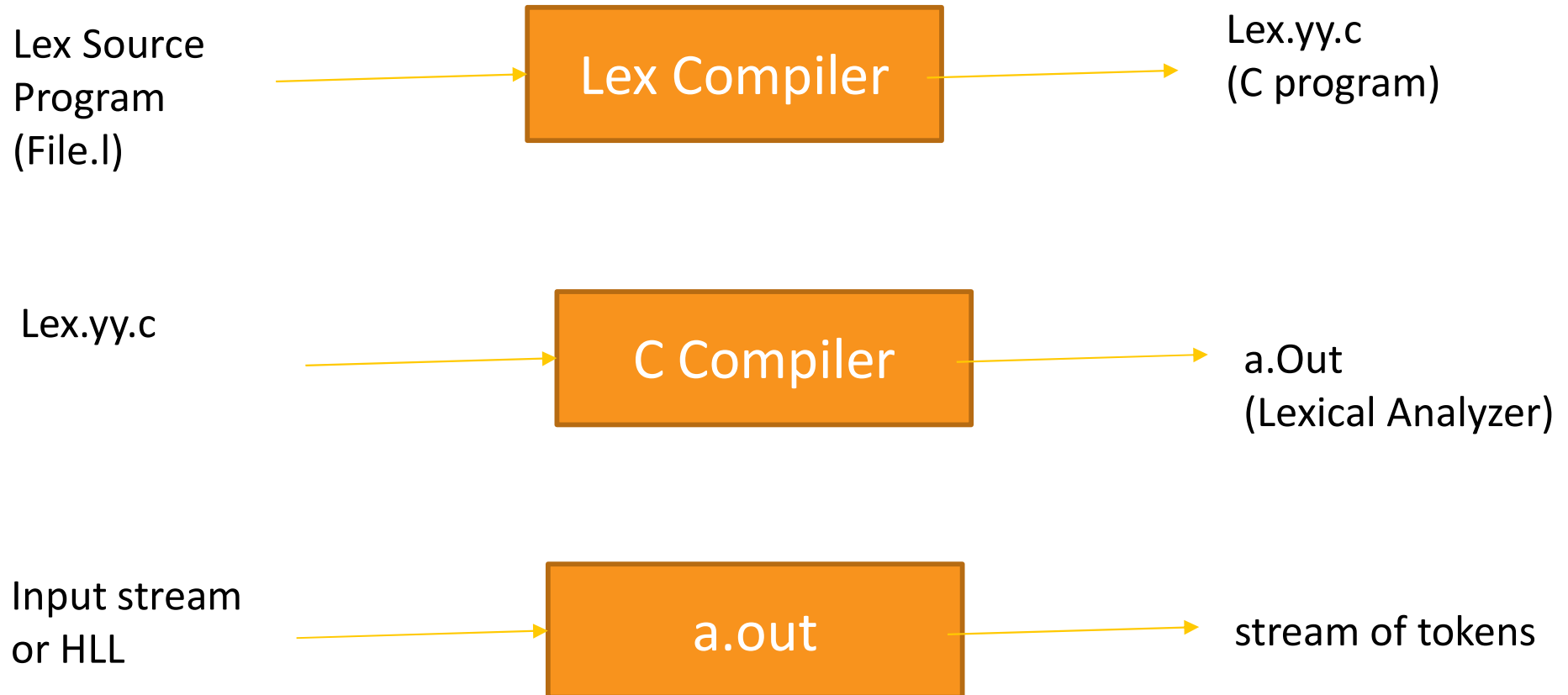

Example of Tokens, Patterns and Lexemes

Token	Pattern	Lexeme
Keyword	characters i f	if
Keyword	characters e l s e	else
Operator	< or > or <= or >= or == or !=	<=, !=
Identifier	letter followed by letters and digits	pi, score, D2
Constant	any numeric constant	3.14

- **Patterns-** Rule that check Lexeme is valid token or not.
or
- Pattern is a rule describing all those lexemes that can represent a particular token in a source language.
- **Lexeme-** A sequence of characters in the source program that is matched by the pattern for a token.
or
- A sequence of input characters that comprises a single token is called a lexeme.

- Lex is a tool which generates Lexical Analyzer .
- Lex written by Mike Lesk and Eric Schridt and described in 1975.

Block diagram of Lex



1. Source Code is in Lex language with file name .l extension.
2. It is given to Lex Compiler which is the Lex tool and produce lex.yy.c
3. C compiler runs this C Code ie lex.yy.c program and produce an output a.out .
4. a.out transforms an input stream into a sequence of tokens.

Structure of LEX

Lex Program is separated into 3 sections by %% delimiters.

{declaration}	%%	{{ declaration of variables including files / libraries}}
{ Translation Rules}	%%	{{Contains rules in form of regular Expression}}
{ Auxiliary functions}		

Ex :

```
% { # include < stdio.h>
    int c = 0 ;
```

```
%}
```

```
%%
```

```
pattern { action }
```

```
%%
```

```
main ( )
```

```
{ }
```

declarartion

{{Contains rules in form of regular Expression}}

Structure of LEX

Lex Program is separated into 3 sections by %% delimiters.

{declaration}	%%	{{ declaration of variables including files / libraries}}
{ Translation Rules}	%%	{{Contains rules in form of regular Expression}}
{ Auxiliary functions}		

Ex :

```
% { # include < stdio.h>
    int c = 0 ;
```

```
%}
```

```
%%
```

```
pattern { action }
```

```
%%
```

```
main ( )
```

```
{ }
```

declarartion

{{Contains rules in form of regular Expression}}

Rule and function used

Rule :

abc : match abc

[a – z] : match small a to z

[a – z]* : match all string in small letters with null or more than 1 character.

[a – z]+ : 1 or more character.

^a means a should come at start.

a\$ means a at end

[0 – 9]+ 1 or more digit

Functions:

YY wrap() : called by lex tool when Input is exhausted. Return 1 if input is finished else 0

YY lex () : reads the input stream and generate tokens according to the regular expression written in rules section.

YY text () : pointer to the input string.

Thank You